# Web Applications

## Express JS

Suhel Hammoud

## What is Express?

Express is a Web Framework built upon Node.js. It is used for building networking services and applications.

## Why Use Express?

- Easy to use functionality for Web Servers
- Open Source and Free
- Easy to extend
- Very performant
- Lots of pre-built packages available

## Table of Contents

## 1. How to Install Express

Initialize a Node.js project:

```
npm init -y
```

Install Express:

```
npm install express
```

## 2. Hello World in Express

```
const express = require('express')
const app = express()

app.get('/', (req, res) => res.send('Hello World!'))

app.listen(3000, () => console.log('Server ready'))
```

Run with:

```
node index.js
```

## 2. Hello World in Express (using Modules)

- Add to package.json

```
{
  "type":"module"
}
```

- Then use import

```
import express from 'express';

const app = express()
```

## 2. Hello World: Explanation

- `require('express')`: import Express
- `express()`: create app instance
- `app.get()`: respond to GET requests
- `res.send()`: send response
- `app.listen()`: start server

## 2. HTTP Verbs in Express

```
app.get('/', (req, res) => { /* */ })
app.post('/', (req, res) => { /* */ })
app.put('/', (req, res) => { /* */ })
app.delete('/', (req, res) => { /* */ })
app.patch('/', (req, res) => { /* */ })
```

## Request and Response Objects

Express provides two key objects:

- `req`: the HTTP request
- `res`: the HTTP response

```
(req, res) => res.send('Hello World!')
```

## Request Parameters Overview

Common `req` properties:

| Property | Description |
|---|---|
| `app` | Express app reference |
| `baseUrl` | Base path of app |
| `body` | Submitted data |
| `cookies` | Cookies sent |
| `hostname` | Host header value |
| `ip` | Client IP |
| `method` | HTTP Method |
| `params` | Route parameters |
| `query` | Query strings |
| `secure` | HTTPS check |

## Retrieve the GET query string paramenter

Express populate the `Request.query` object for us.

- Example:

```
?name=flavio&age=35
```

- Access single query string parameter:

```
req.query.name //flavio
req.query.age //35
```

- Iterate using for in loop:

```
for (const key in req.query) {
console.log(key, req.query[key])
}
```

## retrieve POST query string parameters

### What are POST Query Parameters?

- Sent by HTTP clients (forms, POST requests)
- Not visible in URL (unlike GET)
- Can be sent in different formats:
  - JSON
  - URL-encoded (form data)
  - Multipart (file uploads)

## ▓▓ Accessing POST Data: Request.body

Express provides access to POST data through `req.body`:

```
app.post('/form', (req, res) => {
    const name = req.body.name
    const email = req.body.email
})
```

**But first** - you need middleware to parse the data!

### ▓▓ Middleware 1: JSON Data

For requests with `Content-Type: application/json`

```
const express = require('express')
const app = express()

// Add JSON parsing middleware
app.use(express.json())

// Now you can access JSON data
app.post('/api/data', (req, res) => {
    console.log(req.body)  // Parsed JSON object
    res.json({ received: req.body })
})
```

**Typical use**: REST APIs, mobile apps, fetch/XHR requests

### ▓▓ Middleware 2: URL-Encoded Data (Forms)

For requests with Content-Type: application/x-www-form-urlencoded

```
const express = require('express')
const app = express()

// Add URL-encoded parsing middleware
app.use(express.urlencoded({ extended: true }))

// Now you can access form data
app.post('/form', (req, res) => {
    console.log(req.body)  // Parsed form data
    res.send(`Hello ${req.body.name}`)
})
```

**Typical use**: Traditional HTML forms

## Important Notes

### ✅ Modern Express (v4.16+)

No external modules needed!

```
// Built-in parsers work perfectly
app.use(express.json())
app.use(express.urlencoded({ extended: true }))
```

### ❌ Older Express (< v4.16)

Required body-parser module:

```
// No longer needed for modern Express
const bodyParser = require('body-parser')
app.use(bodyParser.json())
```

## Sending a Response

```
res.send('Hello World!')
```

- Automatically sets Content-Type
- Closes the connection

## Sending a Response

```
res.end(); // Send an empty response
```

```
res.status(404).send('File not found'); // Set HTTP status
```

```
res.sendStatus(404); // Shortcut
```

## Sending JSON Responses

Use `res.json()`:

```
res.json({ username: 'Flavio' })
```

- Converts object/array to JSON

## Managing Cookies

```
res.cookie('username', 'Flavio');
res.clearCookie('username');
```

With options:

```
res.cookie('username', 'Flavio', {
  domain: '.example.com',
  path: '/admin',
  secure: true,
  expires: new Date(Date.now() + 900000),
  httpOnly: true
})
```

## Working with Headers

Access headers:

```
req.headers
req.header('User-Agent')
```

Set headers:

```
res.set('Content-Type', 'text/html')
```

Shortcuts:

```
res.type('json') // application/json
```

## Handling Redirects

Basic:

```
res.redirect('/new-path')
```

Permanent:

```
res.redirect(301, '/new-path')
```

Back:

```
res.redirect('back')
```

## Express Routing

Basic route:

```
app.get('/', (req, res) => { /* ... */ })
```

Named parameters:

```
app.get('/upper/:value', (req, res) =>
  res.send(req.params.value.toUpperCase()))
```

## Regex Routing

Use regex to match paths:

```
app.get(/post/, (req, res) => { /* ... */ })
```

Matches: /post, /post/first, /posting, etc.

## Templates in Express

Set view engine:

```
app.set('view engine', 'pug')
app.set('views', './views')
```

Render view:

```
app.get('/about', (req, res) => {
  res.render('about', { name: 'Flavio' })
})
```

Pug template (views/about.pug):

```
p Hello from #{name}
```

## 📘 Using Handlebars with Express

Handlebars is a popular templating engine supported by Express.

1. Install the required packages:

```
npm install express express-handlebars
```

## Using Handlebars with Express

2. Set up the view engine in your Express app:

```
const express = require('express')
const exphbs = require('express-handlebars')

const app = express()

app.engine('handlebars', exphbs.engine())
app.set('view engine', 'handlebars')
app.set('views', './views')
```

This tells Express to use Handlebars templates located in the `views/` folder.

## Creating and Rendering a Handlebars Template

1. Create a `home.handlebars` file in the `views/` directory:

```
<h1>Hello, {{name}}!</h1>
```

2. Create a route in your Express app to render it:

```
app.get('/', (req, res) => {
  res.render('home', { name: 'Flavio' })
})
```

This will render the HTML with "Hello, Flavio!" in an `<h1>` tag.

You can pass any object to the `res.render()` call and use its properties in the template.

## Express Middleware

Middleware example:

```
app.use((req, res, next) => {
  // Middleware logic
  next()
})
```

Use in a specific route:

```
app.get('/', myMiddleware, handler)
```

## Using Cookie-Parser Middleware

Install and use:

```
npm install cookie-parser
```

```
const cookieParser = require('cookie-parser')
app.use(cookieParser())
```

## Static Assets with Express

Serve static files:

```
app.use(express.static('public'))
```

Files in public/ are served from root path.

## Sending Files to the Client

Download file:

```
res.download('./file.pdf')
```

With custom name and callback:

```
res.download('./file.pdf', 'download.pdf', (err) => {
  if (err) { /* handle error */ }
})
```

## Sessions in Express

Install session middleware:

```
npm install express-session
```

Basic setup:

```
app.use(session({ secret: 'your-secret-key' }))
```

Store data:

```
req.session.name = 'Flavio'
```

## Session Storage Options

Can store session data in:

- Memory (dev only)
- Databases (MySQL, Mongo)
- Caches (Redis, Memcached)

Client receives session ID via cookie.

## Input Validation in Express

Install validator:

```
npm install express-validator
```

Use in route:

```
const { check, validationResult } = require('express-validator')

app.post('/form', [
  check('name').isLength({ min: 3 }),
  check('email').isEmail(),
  check('age').isNumeric()
], (req, res) => {
  const errors = validationResult(req)
  if (!errors.isEmpty()) {
    return res.status(422).json({ errors: errors.array() })
  }
})
```

## Test endpoints

```
curl -X POST http://localhost:3000/form \
  -H "Content-Type: application/json" \
  -d '{"name": "jo", "email": "notanemail", "age": "notanumber"}'
```

## Test endpoints

```
curl -X POST http://localhost:3000/form \
  -H "Content-Type: application/json" \
  -d '{"name": "<script>alert(1)</script>", "email": "notanemail", "age": "notanumber"}'
```

## ▓▓ Sanitize Input

Install sanitizer:

```
npm install express-sanitizer
```

Use in route:

```
const sanitize = require('express-sanitizer')

app.post('/form', [
  sanitize('name').escape(),
  sanitize('email').trim().escape(),
  sanitize('age').toInt()
], (req, res) => {
  const errors = validationResult(req)
  if (!errors.isEmpty()) {
    return res.status(422).json({ errors: errors.array() })
  }
})
```

## ▓▓ Handling Forms

(not covered)

## ▓▓ File Uploads in Express

(not covered)

## Reference

- **Express** (https://expressjs.com/)